SWIFT · XCTEST · RUNTIME

The willThrow Tax

What a test-only runtime hook does to code that throws at volume — and how to get out from under it.

Under Apple's test frameworks, every Swift `throw` is intercepted by a global runtime hook. For ordinary tests it is invisible. For code that throws millions of times — fuzzers, property tests, stress suites — it turns a fast loop slow and quietly leaks memory until the run runs out of RAM. This is the full trace, from the first symptom to the root cause, with reproducible numbers and fixes.

> **Scope, stated up front.** This costs nothing in shipped apps — the hook is inert outside a test process. It never affects correctness. It only bites a narrow workload: throwing at very high volume inside a test. It is a sharp edge, not a broken framework. The one part that is a genuine defect is the memory growth.

~36×	2.1 KB	0×
slower per throw, XCTest vs a plain executable (release)	leaked per throw, never freed until the test method ends	cost in a shipped app — the hook is never installed

---

§1 · DETECTION

How the problem showed up

It surfaced while fuzzing **Kalego**, an encrypted messenger, against the eight parsers that read data off the network. The plan was ordinary: throw hundreds of millions of malformed inputs at each parser and watch for crashes. Three of those parsers *throw* on bad input — by design. So under the fuzzer, throwing was not the rare case. It was the common case, hundreds of millions of times.

The run stalled. One core pinned at **100% for over an hour** while the others sat idle. No crash. No error. No leak warning. Memory looked flat. Everything "worked" — it was just impossibly slow. The kind of failure that wastes an afternoon precisely because nothing announces it.

Because there was no error to read, the next step was to look at what the stuck thread was actually doing — a stack sample of the live process:

```
# find the test process, then sample its stacks
pgrep -f xctest
sample <pid> 5 -f /tmp/sample.txt
```

The sampled stack, top (deepest) to bottom, named the culprit:

```
your loop
  parse(...)
    swift_willThrowTypedImpl            <- fires on every throw
      XCTSwiftErrorObservation...          <- XCTest is watching
        _swift_stdlib_bridgeErrorToNSError <- builds an NSError
          _getErrorUserInfoNSDictionary
            ...CFBasicHashFindBucket, isEqualToString
```

The loop wasn't slow. Something was intercepting *every single throw* and doing real work behind it. That something belonged to XCTest, not to the code under test.

---

§2 · ROOT CAUSE

A global hook the compiler calls on every throw

The Swift runtime exposes a global variable, `_swift_willThrow` — a function pointer. The compiler emits, at every `throw` site, the equivalent of "*if this pointer isn't null, call it.*" In a normal binary the pointer is null, so a throw pays nothing for it. Confirmed directly: in a plain executable the slot reads `0x0`.

Test frameworks install themselves there. Disassembling how Swift Testing registers its handler shows the exact mechanism — an atomic swap into that slot, keeping the previous handler so observers can chain:

```
_swt_setWillThrowHandler:
    ldr   x8, [GOT: __swift_willThrow]   ; address of the slot
    swpal x0, x0, [x8]                    ; atomic swap: install, return the old
    ret
```

So `_swift_willThrow` is not a function — it is a writable pointer that whoever is observing throws swaps their handler into. A symbol scan of the frameworks pins down exactly who installs what:

BINARY	REFERENCES TO WILLTHROW	WHAT IT MEANS
`XCTest.framework`	`0`	The framework itself is clean
`libXCTestSwiftSupport.dylib`	`2 + observer`	The hook lives here (`XCTSwiftErrorObservation`)
`Testing.framework`	`4`	Swift Testing installs its own, too

That last row matters, and it came from the community. In the original Reddit thread, **u/ThatGuy739** not only reproduced the slowdown independently but located the hook in the support dylib and pointed out that `Testing.framework` references `_swift_willThrow` as well — Swift Testing is not immune. Every measurement below that concerns the support dylib and Swift Testing confirms their finding.

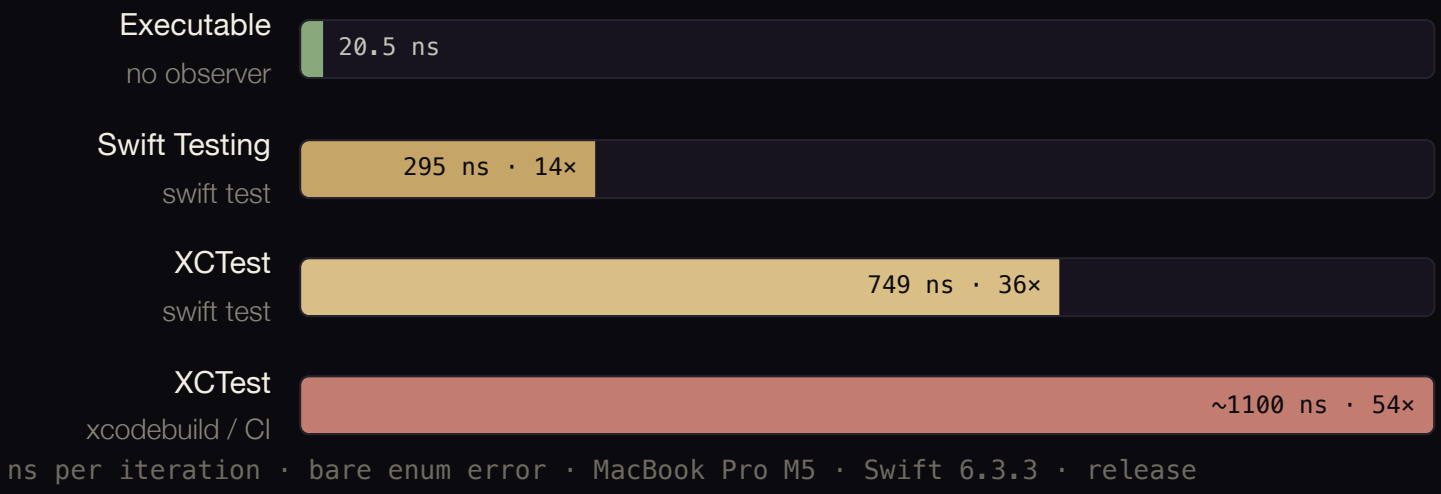
Inside the observer, the per-throw work is: capture the call-stack return addresses, bridge the Swift error to an autoreleased `NSError`, and store the result under an `OSAllocatedUnfairLock` — but only while inside the block that wraps each running test (`observeErrorsInBlock:`). That "only inside the test's block" detail explains a lot of what follows.

---

§3 · MEASUREMENT

The same loop, three places

One function that throws on half its inputs, and the exact same loop compiled into an executable, an XCTest case, and a Swift Testing test. Only the host differs. Release build; time is per loop iteration (half of which throw), so the per-*throw* cost is roughly double.



ns per iteration · bare enum error · MacBook Pro M5 · Swift 6.3.3 · release

The cost is a fixed **~700 ns per iteration (~1.4 µs per actual throw)**. It is *linear* — measured flat from 1M to 8M throws, so the hour-long stall was linear-but-brutal, not a runaway. In a debug build the same absolute ~700 ns is added, but since debug code is already slow it shows up as a milder ~6× rather than ~36×.

---

§4 · THE SECOND PROBLEM

It also leaks memory — this is the real defect

Speed is an annoyance. The memory is the part that is genuinely wrong. Each observed throw under XCTest bridges to an **autoreleased** `NSError`, and XCTest only drains the autorelease pool at the *end* of the test method. Inside one long-running test, nothing is freed:

2,000,000 THROWS IN ONE TEST METHOD	MEMORY Δ	PER THROW
Identical errors	+4,031 MB	2,113 B
Distinct errors	+4,031 MB	2,113 B
Wrapped in `autoreleasepool`	+0 MB	0 B

Two million throws cost **4 GB**, whether the errors are identical or all different. At fuzzing scale — hundreds of millions of throws in a single test — this is not slow, it is an out-of-memory crash. Draining the pool per iteration keeps it perfectly flat, which both proves the mechanism and hands you a fix. There is no good

reason for an observer to let errors accumulate unbounded; this is the one finding that reads as a defect rather than a documented-but-costly design choice.

---

§5 · THE FULL TRACE

Twenty-one findings

Everything measured, grouped by what it does to you. Numbers are per iteration (release, `swift test`) unless noted.

● breaks things ● amplifies the cost ● changes the advice ● ruled out / good news

01 **Unbounded memory growth** — 2.1 KB/throw

Autoreleased NSError per throw, drained only at method end → OOM at scale.

02 **The disarm workaround poisons the process** — footgun

Nulling the slot without restoring it silently disables throw observation for every later test.

03 **The base per-throw cost** — ~1.4 µs

Fixed overhead the observer adds to every throw, independent of build config.

04 **xcodebuild / CI stacks a second observer** — ×1.5

Xcode's runner loads Swift Testing alongside XCTest; the handlers chain. Same test, +51%.

05 **Cost grows with call-stack depth** — up to ×3

The observer captures the whole return-address stack. Throwing deep inside a parser costs more.

06 **Common APIs throw more than once internally** — ×2–3

You pay per internal throw, for throws you never wrote (see §6).

07 **Richer errors cost more** ×1.4

A `CustomNSError` with a userInfo dictionary bridges to a heavier NSError.

08 **Swift Testing is better, not a cure** 295 ns

2.5× cheaper than XCTest and no leak — but still 14× the executable, and loaded under xcodebuild anyway.

09 **Typed throws fix the memory, not the speed** 0 B · 730 ns

`throws(E)` leaks 0 B/throw (vs 2.1 KB) but still costs ~730 ns under XCTest.

10 **Off the test thread, the observer is nearly free** 34 ns

Its work only happens inside the test's own block; background-thread throws bail in the null-check.

11 **Async tests leak identically** 2.1 KB/throw

async is not a mitigation; the pool still doesn't drain mid-loop.

12 **Typed throws are 4× faster — only in the executable** 5.6 ns

5.6 ns vs 20.5 ns with no observer; the observer erases the advantage.

13 **Not superlinear** linear

Per-throw cost is flat from 1M to 8M throws. It's brutal, but it doesn't accelerate.

14 **Not MainActor hops** ruled out

async 596 ns, actor 641 ns — the observer dominates; the actor hop adds only ~45 ns.

15 **No lock contention across threads** scales

Background throwers scale linearly 1→8 threads; the observed path is single-threaded by nature.

16 **Your error's description is not called per throw** `0 calls`

Measured 0 calls over 1,000,000 throws; the string is only built for errors that escape as a failure.

17 **Mixing XCTest and Swift Testing is safe** `no clobber`

Each framework scopes its handler to its own tests; neither clobbers the other.

18 **Disarming does not hide failures** `safe-ish`

XCTUnwrap and assertions record through a separate path; they still fail red.

Three more are the boundary conditions of the fix in §7 — the disarm trick is *process-global and non-atomic* (a throw on another thread during the window is missed), *relies on an internal symbol* that may change between toolchains, and would meet pointer authentication if it tried to *install* a handler rather than clear one. And two remain identified but conditional: an `os_log` emitted per throw only if error-level logging is enabled for the test subsystem, and Instruments signposts only with Instruments attached.

---

§6 · THE MULTIPLIER

You pay for throws you never wrote

A single call into a common Foundation API can throw more than once internally. Each internal throw is observed. Counted directly, by installing a counting handler and calling each API once:

ONE CALL TO…	INTERNAL THROWS
a hand-written `throw` (control)	1
`JSONDecoder.decode` on malformed input	2
`FileManager.contentsOfDirectory` (missing)	2
`FileManager.attributesOfItem` (missing)	3
`NSRegularExpression` / `Regex`, bad pattern	1
`Int("abc")`, `URL(string:)` — return nil	0

So a test that decodes malformed JSON pays roughly double, and one that stats missing files pays triple — for throws inside the standard library. Measured end to end, `JSONDecoder` on bad input runs at 565 ns in an executable and **3,467 ns under XCTest — 6×**. This is why the problem isn't only a fuzzer's problem: any test that exercises error paths at volume pays it. And the amplifiers compound:

`1.4 µs base` × `3 stack depth` × `2 API` × `1.5 CI` = ~12 µs per call

worst-case composition: a deep throw, in a decoder, under CI

---

§7 · FIXES

What actually gets you out

Ranked. The first two are the safe answers; the rest are situational.

APPROACH	SPEED	MEMORY	CAVEAT
Move the hot loop to an executable (a CLI / benchmark target)	full	full	Best for fuzzing and stress. No observer at all.
Run the loop on a background thread inside the test	full	full	Throws there aren't attributed to the test — fine for a fuzzer.
Wrap each iteration in `autoreleasepool`	still slow	full	Stops the OOM even if you keep the loop in XCTest.
Use typed throws `throws(E)`	unchanged	full	Removes the leak; CPU cost stays.
Return `nil` instead of throwing on the hot path	full	full	Changes the API; right when the "error" is really a common case.
Disarm the observer around the loop	full	full	**Sharp. Must restore, or you poison later tests.**

The last one is the clever option and the dangerous one. The observer lives in a writable slot, so you can clear it around a hot loop and put it back. It works — 34× — but if you forget to restore it (an early return, a thrown error, a crash mid-loop), throw observation stays dead for the rest of the process. If you use it, gate it behind `defer` and keep the window tiny:

```
// Clears the willThrow observer for the duration of `body`, then restores it.
// Relies on an internal runtime symbol — pin it to a toolchain and test it.
func withThrowObserverDisabled<R>(_ body: () -> R) -> R {
    let slot = dlsym(dlopen(nil, RTLD_NOW), "_swift_willThrow")?
        .assumingMemoryBound(to: UnsafeRawPointer?.self)
    let saved = slot?.pointee
    slot?.pointee = nil
    defer { slot?.pointee = saved }   // or every later test loses observation
    return body()
}
```

If you only need the memory safe and can't leave XCTest, the honest minimum is one line: wrap the loop body in `autoreleasepool { }`. If you're fuzzing, don't fight it — put the loop in an executable.

§8 · VERDICT

How serious is it, really

Calibrated, without the hype: **a narrow, test-time-only sharp edge.** It costs nothing in production — the hook is never installed in a shipped binary. It never affects correctness. It only hurts code that throws at very high volume inside a test, which is a small population — but a real one, and one that gets no warning when it happens.

Of everything here, one item is a genuine defect rather than an expensive-by-design trade-off: the **unbounded memory growth**, which turns a slow test into an out-of-memory crash and is fixable by draining the pool. The rest is the runtime doing something useful — recording where an unexpected error was thrown — without a documented cost or a cap. If you never throw in a tight loop under test, none of this will ever touch you.

§9 · REPRODUCE IT

Sixteen lines

A function that throws on half its inputs, and the identical loop in two hosts. Run both; watch the executable finish in well under a second and the test crawl.

```
// Lib.swift
public enum ParseError: Error { case malformed }
@inline(never) public func parse(_ x: Int) throws -> Int {
    if x & 1 == 0 { throw ParseError.malformed }
    return x
}

// the loop — identical in the executable and in the XCTest case:
for i in 0..<20_000_000 { _ = try? parse(i) }
```

The reproducible bench used for every number above — all twenty-one experiments as separate targets — runs the same loop across an executable, an XCTest target and a Swift Testing target, plus the symbol and memory probes. Check the figures on your own machine; that is the whole point.

§10 · CREDITS

Who found what

This started as one person's slow afternoon and got sharper because other people pushed on it in public. That's worth naming.

Yassin Daoud
`u/Next-Manufacturer375 · github/MagicYassin`
Found it while fuzzing Kalego; ran the full investigation and the twenty-one experiments.

u/ThatGuy739
`r/swift`
Reproduced it independently on an M5, located the hook inside `libXCTestSwiftSupport.dylib`, and flagged that Swift Testing references `_swift_willThrow` too — the thread's most valuable contribution.

u/Dry_Hotel1100
`r/swift`
Pushed the hardest questions — is a test's elapsed time even a valid criterion, and could this be MainActor hops? — which is exactly what forced the rule-out experiments in §5.

`MacBook Pro M5 · Swift 6.3.3 · macOS 26 · Xcode 26.6 · release unless noted`

Measure it yourself before you believe any of it.